MSBA 305: Data Processing Framework

Spring 2025/2026

Dr. Ahmad El-Hajj

Course Project: Implementing a full data pipeline

Prelude

In the age of AI-assisted coding, *anyone can generate code*. What separates a data professional from a prompt engineer is the ability to **make and defend architectural decisions**. This is of notable importance for business analysts whose input can reduce costs or drive them incredibly high.

1 Project Overview

You will implement a **complete data processing pipeline**, applying the full data lifecycle from acquisition to storage, transformation, and query-ready analytics. The goal is to work with real-world datasets while demonstrating mastery of data architecture principles.
Think of yourselves as a **Data Architecture Team** at a consulting firm. A client has approached you with a data challenge in their domain. Your job is to:

1. **Identify and evaluate data sources** that address their business needs
2. **Design and implement a data pipeline** from ingestion to query-ready storage
3. **Justify every architectural choice** with alternatives considered and rejected
4. **Deliver working queries and insights** that demonstrate the pipeline's value

Note: This project covers the pipeline up to *query-ready data*. Predictive analytics and machine learning are out of scope, though you're encouraged to explore them if time permits.

2 Domain Selection & Data Requirements

Your team selects the application domain. The domain must meet these requirements:

- **Minimum 3 different data sources** (e.g., transactional + external API + reference data)
- **At least 2 different data formats** (CSV, JSON, XML, API response, etc.)
- **Realistic data quality challenges** (missing values, inconsistencies, duplicates)
- **Clear analytical objectives** (questions the pipeline should answer)

Technology Options: Suggested Data Sources

- **Open Government Data:** Data.gov, EU Open Data Portal, World Bank Open Data
- **Industry Datasets:** Kaggle, Google Dataset Search, UCI ML Repository
- **APIs for Real-Time Data:** OpenWeather API, Yahoo Finance API, News APIs
- **Web Scraping:** BeautifulSoup, Scrapy, Selenium (ensure ethical/legal compliance)

Example domains (you are not limited to these):

Domain	Example Data Sources
E-commerce	Transactions (CSV), Product catalog (JSON), Customer reviews (API)
Smart City	Traffic sensors (streaming), Weather API, Events calendar (XML)
Financial Services	Stock prices (API), Company filings (PDF/XML), News sentiment (API)
Healthcare Ops	Appointments (SQL export), Equipment logs (JSON), Staffing (CSV)
Supply Chain	Supplier data (CSV), Shipping APIs, Demand forecasts (Excel)
Media & Entertainment	Streaming logs, User engagement (JSON), Content metadata (API)

3 The Decision Documentation Requirement

For **every major decision** in your architecture, you must document:

1. **What you chose**
2. **At least 2 alternatives** you considered
3. **Why you rejected** each alternative (with specific reasoning)
4. **Trade-offs** you are accepting with your choice

Example Decision Documentation:

Component	Content
Decision	Primary storage: MongoDB
Alternatives	(1) PostgreSQL with JSONB columns, (2) Neo4j graph database
Why Rejected	(1) PostgreSQL: Schema evolution requires migrations; our product catalog has 15+ optional fields that change quarterly. (2) Neo4j: Our queries are document-centric (find products), not relationship-heavy; graph traversal adds latency without benefit.
Trade-offs	Accepting: No native joins, eventual consistency. Mitigating with denormalized documents and application-level validation.

4 Pipeline Components & Technical Requirements

4.1 Data Acquisition & Ingestion

Objective: Obtain real-world data and document your ingestion strategy.

Technical Tasks:

- Implement data collection scripts (API calls, web scraping, file downloads)
- Design ingestion pipeline (batch vs. streaming vs. hybrid)
- Implement error handling and retry logic
- Document data source characteristics (volume, velocity, format)

Technology Options: Technology Options for Ingestion

- **Batch:** Python (requests, pandas), Apache Airflow for orchestration
- **Streaming (optional):** Apache Kafka, Spark Streaming, AWS Kinesis
- **Web Scraping:** BeautifulSoup, Scrapy, Selenium

Required Deliverables:

- Data source appraisal table (see Section 4.1.1)
- Working ingestion scripts with documentation
- Justification for batch vs. streaming decision

4.1.1 Data Source Appraisal (Required for Each Source)

For each data source you use, complete the following appraisal:

Dimension	What to Document
Source & Collection	Origin, ownership, collection mechanism, access method
Structure & Format	File format, schema (rigid/flexible), nested structures, encoding
Volume & Velocity	Current size, growth rate, update frequency, latency requirements
Quality Assessment	Completeness %, accuracy, consistency issues (with evidence)
Risks & Limitations	Biases, missing patterns, ethical/legal concerns, schema drift risks

4.2 Data Storage & Integration

Objective: Design and implement storage architecture appropriate for your data types.

Technical Tasks:

- Select and justify database technology (relational, document, graph, columnar)
- Design schema/document structure with diagrams
- Implement data integration logic (combining multiple sources)
- Define indexing and partitioning strategy

Technology Options: Storage Technology Options

- **SQL Databases:** PostgreSQL, MySQL for structured data with ACID requirements
- **Document Stores:** MongoDB, Couchbase for semi-structured, schema-flexible data
- **Graph Databases:** Neo4j for relationship-heavy data (social networks, knowledge graphs)
- **Columnar Stores:** Apache Cassandra, ClickHouse for time-series, high-volume analytics
- **Cloud Storage/Processing:** AWS S3, Google Cloud Storage, Azure Blob, Microsoft Fabric

Required Deliverables:

- Database schema or document structure (with ER diagram or equivalent)
- Justification for storage technology choice (with alternatives)
- Sample queries demonstrating data retrieval

4.3 Data Cleaning & Transformation

Objective: Ensure data quality and consistency for reliable analytics.

Technical Tasks:

- Handle missing values (deletion, imputation, flagging) with justification
- Remove or merge duplicates with documented deduplication logic

- Standardize formats (dates, currencies, units, categorical values)
- Handle outliers and anomalies with documented approach
- Implement data validation rules

Technology Options: Cleaning & Transformation Tools

- **Python:** pandas, numpy for local processing; great-expectations for validation
- **Distributed:** PySpark, Databricks for large-scale transformation
- **SQL-based:** dbt (data build tool) for transformation-as-code

Required Deliverables:

- Cleaning code with comments explaining each transformation
- Data quality report: before/after statistics
- Documentation of cleaning decisions and their rationale

4.4 Data Processing for Analytics

Objective: Process data using appropriate computation paradigms and extract initial insights.
Technical Tasks:

- **Batch Processing:** Aggregate historical trends, compute statistics, build derived tables
- **Stream Processing (optional):** Real-time aggregations, windowed computations
- **EDA:** Summary statistics, distributions, correlations, initial visualizations
- Decide on local vs. distributed processing (justify based on data volume)

Technology Options: Processing Technology Options

- **Local Processing:** pandas, polars for datasets that fit in memory
- **Distributed Batch:** Apache Spark, Databricks, Dask for large-scale processing
- **Stream Processing:** Kafka Streams, Spark Streaming, Flink for real-time pipelines

Required Deliverables:

- Processed dataset (structured, ready for querying)
- Jupyter notebooks or scripts demonstrating transformations
- EDA visualizations with interpretation

4.5 Data Querying & Insights

Objective: Extract meaningful insights that demonstrate the pipeline's analytical value.
Technical Tasks:

- Define analytical questions your pipeline answers
- Implement queries using appropriate query language (SQL, MQL, Cypher)
- Optimize query performance (indexing, query plans)
- Interpret results with business context

Technology Options: Query Languages by Database Type

- **SQL Databases:** Standard SQL - JOINs, aggregations, window functions, CTEs
- **MongoDB:** Aggregation pipeline -`$match`, `$group`, `$lookup`, `$project`
- **Neo4j:** Cypher - MATCH, WHERE, RETURN, path patterns

Required Deliverables:

- **Minimum 5 analytical queries** with increasing complexity
- Query results with interpretation (what does this tell us?)
- Performance notes (execution time, optimization applied)

4.6 Visualization & Reporting

Objective: Present findings in an interpretable, actionable format.

Technical Tasks:

- Create visualizations that tell a story from your data
- Build a dashboard or visualization report
- Summarize key patterns, trends, and anomalies

Technology Options: Visualization Tools

- **Python Libraries:** matplotlib, seaborn, plotly, altair
- **Dashboards:** Streamlit, Dash, Gradio (Python-based)
- **BI Tools:** Power BI, Tableau, Looker (very useful for students selecting Google Cloud Platform)

Required Deliverables:

- Dashboard or visualization report with key insights
- Written interpretation of visualizations (not just charts)

4.7 Security, Governance & Archival Strategy

Objective: Define data management policies for production readiness.

Technical Tasks:

- Data classification (what is sensitive? PII? confidential?)
- Access control design (who can read/write what?)
- Data retention and lifecycle policies
- Backup and disaster recovery strategy
- Ethical considerations and bias documentation

Required Deliverables:

- Data governance documentation (classification, access, retention)
- Cloud vs. local storage trade-off analysis

5 Submission Requirements

Deliverable	Description	Weight
Architecture Report	Complete documentation of all decisions, alternatives, justifications. Include architecture diagrams, data flow, and schema designs. Proper referencing required.	65%
Code Repository	GitHub repository with: well-documented scripts, Jupyter notebooks, sample data, README with setup instructions. If AI used, include prompts with generated output.	15%
Presentation	15–20 min presentation including Q&A. Summarize key insights and technical decisions. Every team member must speak and answer questions.	20%

6 Presentation guidelines

The presentation is where we evaluate your understanding. You will present in front of a judging panel who will evaluate your work objectively. Main points include the completeness, cohesiveness and validity of your approach. Remember that storytelling is key. The jury will judge you solely based on your presentation. Try to limit the presentation to **a maximum of 15 minutes** with **5 min for Q&A**
Some questions could be as follows:

- *"Why MongoDB instead of PostgreSQL?"*
- *"What happens if this data source goes down?"*
- *"Walk me through this aggregation pipeline."*
- *"What would change if your data volume 10x-ed?"*
- *"This data source has bias. How did you handle it?"*

7 Use of Generative AI

You are **permitted and encouraged** to use generative AI tools (such as ChatGPT, Claude, Copilot, etc.) to assist with this assignment. However, the following requirements are **mandatory**:

AI Usage Documentation

- You **must** document every instance where you used AI assistance
- For each AI interaction, include:
  - The exact prompt you provided to the AI
  - A summary of the AI's response
  - How you incorporated the AI's suggestions into your work
  - Any modifications you made to the AI's output

Required Documentation Format

Include an "AI Usage" section in your report with entries like:

```
## AI Interaction [Number]
**Task:** Cleaning Pipeline
**Prompt:** "How do I parse multiple date formats in Python?"
**AI Response Summary:** Suggested using dateutil.parser with
    try-except for multiple format attempts.
**Implementation:** Used the suggested approach for date
    standardization in my parse_date() function.
**Modifications:** Added additional date formats specific to
    the dataset (DD-MM-YYYY, YYYY/MM/DD).
```

Academic Integrity Requirements

- You **must understand and be able to explain** every line of code you submit
- Direct copy-pasting of AI-generated code without understanding is considered **academic dishonesty**
- The analytical interpretation, architecture justification, and business impact assessment must reflect your **personal understanding**
- AI tools count as sources and must be cited in your references section

Warning: Submitting AI-generated analysis without understanding, or copying from classmates, constitutes academic dishonesty under the AUB Student Code of Conduct.

8 Timeline & Logistics

Date	Milestone
Monday February 23, 2026	Team formation + Domain selection. Submit 1-page proposal with team members, domain, and initial data sources to ae37@aub.edu.lb (Mandatory)
Monday March 16, 2026	Data source appraisal complete. Submit data appraisal report. (Optional)
Monday April 6, 2026	Architecture draft review (optional feedback session during office hours)
Monday April 20, 2026	Final submission via Moodle: Report + GitHub repository
Saturday April 25, 2026 (Online)	Presentations (Time to be confirmed)

Team Requirements:

- Maximum 5 students per team
- All team members must contribute to every phase

Good luck!